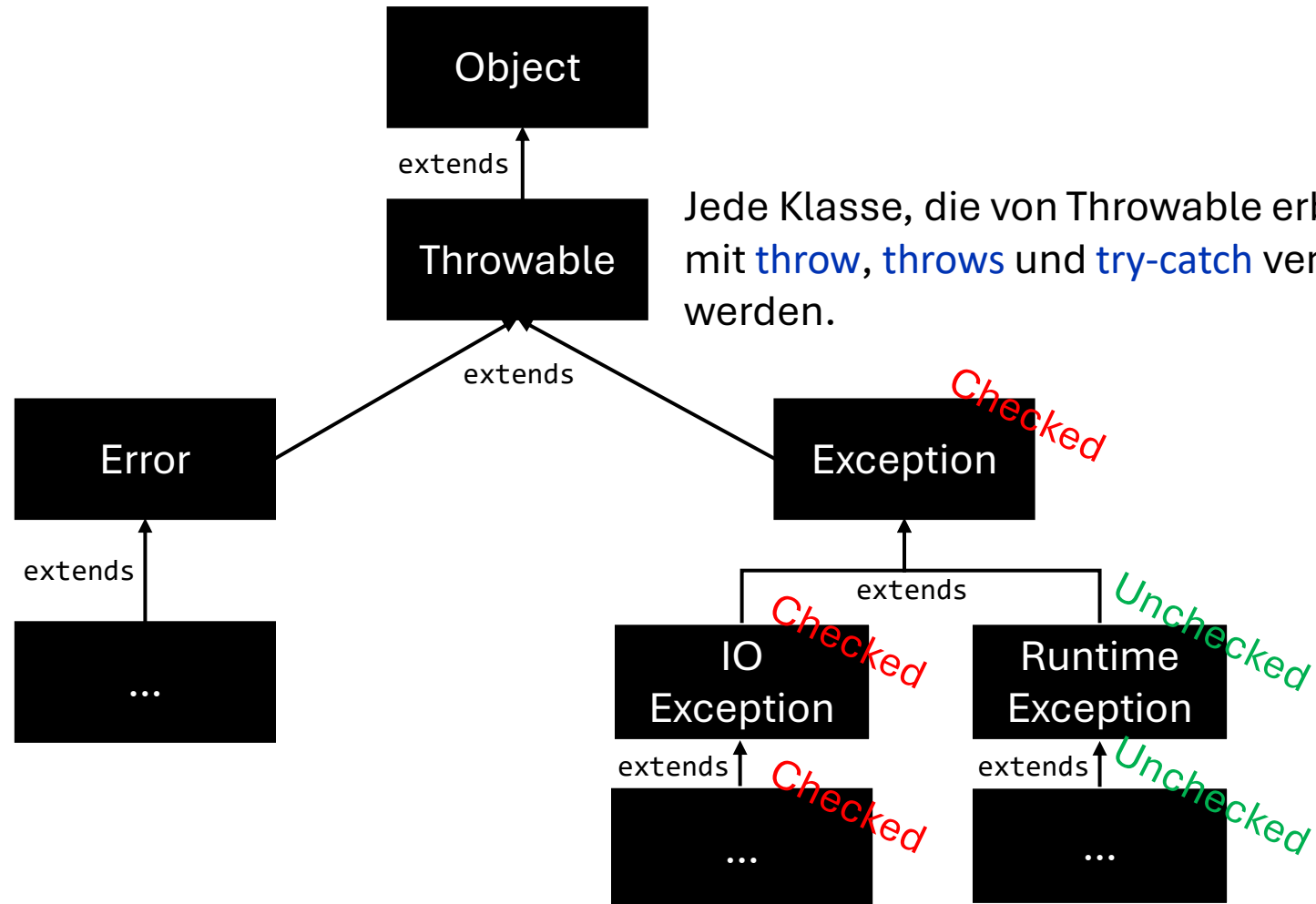




Exceptions, Optional und Null Safety

Exceptions in Java

"Ein Error weist auf ernsthafte Probleme hin, die eine sinnvolle Anwendung nicht abfangen sollte. Error finden unter abnormalen Bedingungen statt, die niemals auftreten sollten."
Beispiel: Hardwarefehler.



Checked Exception

Eine Checked Exception muss **vorm Kompilieren behandelt** werden durch:

- try-catch-Block
- Methodensignatur (leitet das "Problem" an den Aufrufer weiter).

Beispiele:

Exception und alle seine Ableitungen wie z. B. **IOException** oder **SQLException** (außer RuntimeException).

Unchecked Exception

Eine Unchecked Exception muss **nicht vorm Kompilieren behandelt** werden. Sie tritt erst zur **Laufzeit** auf.

Unchecked Exception in der Methodensignatur ist **optional**.
Leitet das "Problem" transparent weiter an den Aufrufer.

Beispiele:

RuntimeException und alle seine Ableitungen wie z. B.
NoSuchElementException, **NullPointerException** oder
ArrayIndexOutOfBoundsException.

Checked oder Unchecked Exception?

Java basiert auf der *Java Virtual Machine* (JVM) und ist eine der wenigen Programmiersprachen, die Checked Exceptions unterstützt. Weitere Sprachen mit Checked Exceptions sind Kotlin (JVM), Scala (JVM) und Objective-C.

Obwohl sich mögliche Fehler bereits durch den Compiler vorab prüfen lassen, haben sich Checked Exceptions kaum in anderen Programmiersprachen etabliert. Stattdessen wird oft die Flexibilität bevorzugt, Unchecked Exceptions nicht immer behandeln zu müssen.

Eigene Exceptions erstellen

In Java sind eine eigene „Exception“ eine normale Klasse, die von der Klasse `Exception` oder einer ihrer Unterklassen erbt. Der Name der Klasse entspricht dabei dem Namen der Exception.

Je nachdem, ob von der Klasse `Exception` oder von der Klasse `RuntimeException` geerbt wird, handelt es sich um eine Checked oder Unchecked Exception.

Der Aufbau ist in der Regel immer derselbe: ein Konstruktor, der eine Fehlernachricht als Parameter erhält und diese an den übergeordneten Konstruktor (`super`) weitergibt.

```
class MyCheckedException extends Exception {  
    public MyCheckedException(String message) {  
        super(message);  
    }  
}
```

```
class MyUncheckedException extends RuntimeException {  
    public MyUncheckedException(String message) {  
        super(message);  
    }  
}
```

Wann eigene Exceptions erstellen?

Wann erstellt man eigene Exceptions?

- Fachspezifische Fehler: Wenn ein Fehlerfall sehr spezifisch für die Fachlogik (Domäne) ist und durch Standard-Exceptions nicht ausreichend ausgedrückt wird. Beispiel: `InsufficientFundsException` in einer Banking-Anwendung.
- Bessere Lesbarkeit und Wartbarkeit: Eigene Exception-Namen machen den Code verständlicher und erleichtern die Fehlerbehandlung.
- Zusätzliche Informationen: Wenn die Exception neben der Fehlermeldung weitere Daten enthalten soll (z. B. fehlerhafte Werte, Statuscodes).
- API-Design: Wenn eine öffentliche Schnittstelle bereitgestellt wird und spezifizierte, dokumentierte Fehlerarten verwendet werden sollen.

Wann verwendet man vorhandene Exceptions?

Allgemeine Fehlerfälle: Wenn der Fehler in eine Standardkategorie passt.

- `IllegalArgumentException`: Für ungültige Parameterwerte.
- `NoSuchElementException`: Wenn ein erwartetes Element fehlt (z. B. in Collections).
- Wenn eine Standard-Exception den Fehler ausreichend beschreibt und keine zusätzlichen Informationen nötig sind.
- Konsistenz mit Java-Standards: Vermeidet unnötige Komplexität und erleichtert die Zusammenarbeit mit anderen Entwicklern.

Globales Exception Handling?

Frage: Warum nicht alle Exceptions global "ganz oben" in der main-Methode abfangen?

```
public static void main(String[] args) {  
    try {  
        // start application  
        new MainView().show();  
        // run app ...  
    } catch (Exception e) {  
        System.err.println(e.getMessage());  
    }  
}
```

Antwort: In der Regel möchte man eine Exception möglichst lokal abfangen und behandeln (Fail Fast Prinzip), z. B. weil der Fehler im Dialog-Fenster angezeigt werden soll und nicht im Hauptfenster. Es ist zudem oft sinnvoller, sofern möglich, Exceptions lokal zu behandeln, damit sie nicht überall im Softwaresystem auftauchen.

Darüber hinaus sind Exceptions langsam im Vergleich zu if-else-Abfragen, da beim Erzeugen einer Exception jedes Mal ein Call-Stack (Historie der Methodenaufrufe) mit erstellt wird.

Fail Fast Principle



Je später Fehler behandelt werden, **desto schwieriger** lässt sich die **Fehlerursache** im Nachhinein ermitteln.

Aus diesem Grund besagt das Fail Fast Principle:

- Fehler so früh wie möglich erkennen.
- Fehler so früh wie möglich zurückmelden.
- Fehler möglichst lokal behandeln und nicht weitergeben.

Ziel: Einfache Diagnose und Debugging.

Fail Fast Principle: Beispiel (Preconditions)

Prüfe **zu Beginn** einer **Methode** oder eines **Konstruktors**, ob die Parameter **valide** sind.
Beispiel:

```
Optional<Invoice> calculateByCustomer(String name) {  
    // 1. do validity checks first (preconditions).  
    if(name == null || name.isBlank()) {  
        throw new IllegalArgumentException(name); // or use Optional<T> instead of Exception (see upcoming slides).  
    }  
  
    // 2. then do the actual operation with valid parameters.  
    // query customer by name...  
}
```

Die Prüfung am Anfang der Methode nennt man auch **Precondition**: die Methode prüft zunächst alle Parameter auf Gültigkeit. Ist ein Parameter ungültig, wird die Methode sofort mit einer Exception abgebrochen, da eine Vorbedingung verletzt wurde und die Methode nicht korrekt ausgeführt werden könnte.

null



Null Problemo



null sollte möglichst vermieden werden, da sonst überall im Code **null**-Checks erforderlich sind:

```
class InvoiceManager {
```

```
    Invoice calculateByCustomer(String name) {
```

```
        var customer = database.findCustomer(name);
```

```
        if (customer != null) {
```

```
            // do calculations with customer and fill result...
```

```
            return new Invoice(42.50);
```

```
        } else {
```

```
            return null; // Der Aurufer von calculateByCustomer muss wieder einen null-Check für die Invoice machen... 🤪
```

```
        }
```

```
    }
```

```
}
```

Primitive statt null

Primitive (`byte`, `short`, `int`, `long`, `float`, `double`, `boolean`, `char`) sind, soweit möglich, vorzuziehen, da diese niemals `null` sein können. 👍

`int number = null;` // Nicht kompilierbar. Nur Objekte können null sein.

Objekte können dagegen `null` sein, wie z. B. bei den Wrapper-Klassen Integer, Double, Character usw.

```
class MathUtil {  
    static double square(double number) { // Rückgabewert und Parameter kann niemals null sein, kein null-Check notwendig!  
        return number * number;  
    }  
  
    static Double square(Double number) { // Rückgabewert und Parameter kann null sein, null-Check notwendig!  
        return number * number;  
    }  
  
    static void main(String[] args) {  
        System.out.println(square(null)); // Der Compiler wählt hier die Double-Methode aus, weil nur diese Methode null-Werte akzeptiert.  
    }  
}
```

Exception statt null

Eine Möglichkeit, `null` zu vermeiden, wäre eine `RuntimeException` zu werfen.

```
interface Database {  
    Customer findCustomer(String name) throws DatabaseException; // DatabaseException erbt von RuntimeException  
}  
  
class InvoiceManager {  
    public Invoice calculateByCustomer(String name) {  
        var customer = database.findCustomer(name); // wirft eine RuntimeException.  
        // do calculations with customer and fill result...  
        return new Invoice(42.50);  
    }  
}
```

Vorteil: Die `RuntimeException` kann ignoriert werden.

Nachteil: Die Exception ist "versteckt". Sie ist häufig nur im JavaDoc dokumentiert und kann daher übersehen werden. Das kann dazu führen, dass ein `try-catch` vergessen wird (lokal oder global), und die App zur Laufzeit abstürzen kann. Das sollte nicht passieren!

Optional<T>



Optional<T> ist eine Wrapper-Klasse: Ein Optional<T>-Objekt enthält ein anderes Objekt. Im Grunde funktioniert ein Optional<T> wie eine Collection<T> mit maximal einem Element (Optional<T> ist allerdings nicht Iterable).

Mit Optional<T> lässt sich in der Programmiersprache ausdrücken, dass ein Objekt eventuell nicht vorhanden ist (anstatt **null** zu verwenden).

Durch die `ifPresent`- und `orElse`-Methoden in Optional<T> kann das enthaltene Objekt herausgeholt und gleichzeitig eine Alternative codiert werden, falls das Objekt nicht vorhanden (**null**) ist. Dadurch lassen sich if-else-Abfragen vermeiden. 🤖

Optional<T> erzeugen

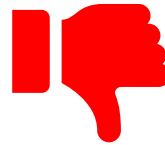
Es gibt in der Klasse Optional<T> folgende statische Factory-Methoden, um ein Optional-Objekt zu erzeugen:

1. `return Optional.empty();` *// ein leeres Optional-Objekt, quasi der Ersatz für null*
2. `return Optional.of(customer);` *// wirft NullPointerException wenn customer == null*
3. `return Optional.ofNullable(customer);` *// i. d. R. die Methode der Wahl.
Falls customer == null, return empty Optional, sonst ein Optional mit dem customer-Objekt.*

Optional<T> verwenden

Einige Methoden aus [Optional<T>](#):

~~boolean isEmpty()~~ // *möglichst vermeiden*
~~boolean isPresent()~~ // *möglichst vermeiden*
~~T get()~~ // *möglichst vermeiden*



T orElse(T other)
T orElseGet(lambda)
T orElseThrow()
T orElseThrow(lambda)



void ifPresent(lambda)
void ifPresentOrElse(lambda, lambda)

Optional<U> map(lambda) // *Optional<T> kann .stream() und somit alle Stream-Methoden.* 🤖

Optional<T> or...



Man gewinnt mit den Optional-Methoden `isPresent` und `get` nichts – man hätte genauso gut `null` zurückgeben können statt ein `Optional`, weil man wieder eine `if`-Abfrage macht. Beispiel:



// null-Variante:

```
Customer customer = db.findCustomer(name);  
if (customer != null) { // null-Check  
    IO.println(customer.name());  
} else {  
    IO.println("NoName");  
}
```



// Optional-Variante mit isPresent und get:

```
Optional<Customer> customer = db.findCustomer(name);  
if (customer.isPresent()) { // quasi null-Check  
    IO.println(customer.get().name());  
} else {  
    IO.println("NoName");  
}
```



*// orElse gibt entweder den Wert im Optional zurück, wenn dieser existiert; falls nicht, wird ein Default-Wert "NoName" zurückgegeben.
// Optional unterstützt auch die Stream-Methode map. Diese wird hier genutzt, um den name() zu extrahieren, sofern der customer existiert.*

```
IO.println(customer.map(Customer::name).orElse("NoName"));
```

Optional<T> statt `null`

Optional<T> sollte **als Rückgabewert** verwendet werden, falls das Objekt `null` sein könnte.

- Mit Optional kann **explizit in der Sprache ausgedrückt** werden, dass ein Objekt eventuell nicht vorhanden (`null`) sein könnte.
- Mit Optional<T> lässt sich bereits zur Compile-Zeit `null` isolieren und damit NullPointerException vermeiden.

Hinweis: Die Verwendung von Optional<T> für Parameter ist umstritten.

Besser: Methoden-Overloading oder [Varargs...](#)

"API Note: Optional is primarily intended for use as a method return type where there is a clear need to represent "no result," and where using null is likely to cause errors. A variable whose type is Optional should never itself be null; it should always point to an Optional instance."

To optional or not to optional?



Variante 1: Optional

```
Optional<Invoice> calculateByCustomer(String name) {  
    Customer customer = database.findCustomer(name);  
    if(customer == null) {  
        // return Optional.empty();
```

```
        // throw new NoSuchElementException("Customer " + name + " not found.");
```

Variante 2: Exception

```
    }  
    double sum = customer.orders().stream().mapToDouble(order -> order.totalCost()).sum();  
    Invoice invoice = new Invoice(name, sum);  
    return invoice;  
}
```

Was tun, wenn kein customer gefunden wird?



Wann Exception verwenden?

- Exceptions sind in der Regel **langsam in Bezug auf Performanz**.
- Exceptions sind für **außergewöhnliche Situationen** gedacht.
Z. B. I/O-Probleme
- Exceptions werden auch für **Preconditions** verwendet.
- Verwende Exception, wenn es **keine alternative Handlung** gibt.
D. h., es lässt sich nichts weiter machen, außer den Fehler auszugeben.
- Verwende Exceptions **nicht**, um den **Kontrollfluss zu steuern**.
D. h., wenn sich etwas leicht mit if-else prüfen lässt, dann nutze if-else.

```
// do not do this:  
Student student = new Student("Sara");  
try {  
    return student.name(); // student can be null  
} catch (NullPointerException e) {  
    return "";  
}
```



```
// do this instead:  
if(student == null) {  
    return "";  
}  
return student.name();
```

```
// or even better use an Optional<Student>:  
return student.map(Student::name).orElse("");
```



Wann Optional<T> verwenden?

- Optional<T> sollte nur **für Rückgabewerte von Methoden** verwendet werden.
- **Ausnahme:** Wenn eine **Methode ein Iterable (Collection) zurückgibt**, ist es in der Regel besser, entweder **die Ergebnisliste** zurückzugeben oder **eine leere Liste** (z. B. bei Suchanfragen).
 - Ausnahme von der Ausnahme:
 - Eine Ergebnisliste darf nicht leer sein (z. B., um Division durch 0 zu vermeiden).
 - Es soll eine explizite Fehlermeldung an den User kommuniziert werden.
- Verwende Optional<T> **nicht für Parameter!**
Verwende stattdessen Method-Overloading oder Varargs.

Optional<T> Parameter vs Method Overloading

Optional-Parameter (Vermeide):

```
Optional<Customer> findCustomer(Optional<String> customerName) {  
    try {  
        String fileContent = Files.readString(Path.of(CUSTOMERS_DB_FILE));  
        return parseCustomer(customerName.orElse("defaultCustomer"), fileContent);  
    } catch (IOException e) {  
        throw new DatabaseException("File not found: " + CUSTOMERS_DB_FILE);  
    }  
}
```



Method Overloading (Best Practice):

```
Optional<Customer> findCustomer() { // Method-Overloading von findCustomer ohne customerName  
    return findCustomer("defaultCustomer"); // Verwende Implementation von findCustomer wieder. #DRY  
}
```

```
Optional<Customer> findCustomer(String customerName) {  
    try {  
        String fileContent = Files.readString(Path.of(CUSTOMERS_DB_FILE));  
        return parseCustomer(customerName, fileContent);  
    } catch (IOException e) {  
        throw new DatabaseException("File not found: " + CUSTOMERS_DB_FILE);  
    }  
}
```

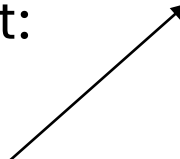


Optional<T> und Exception

Häufig werden Optional<T> und Exception auch kombiniert:

```
public Optional<Customer> findCustomer(String name) throws DatabaseException {  
    return findCustomer("defaultCustomer");  
}
```

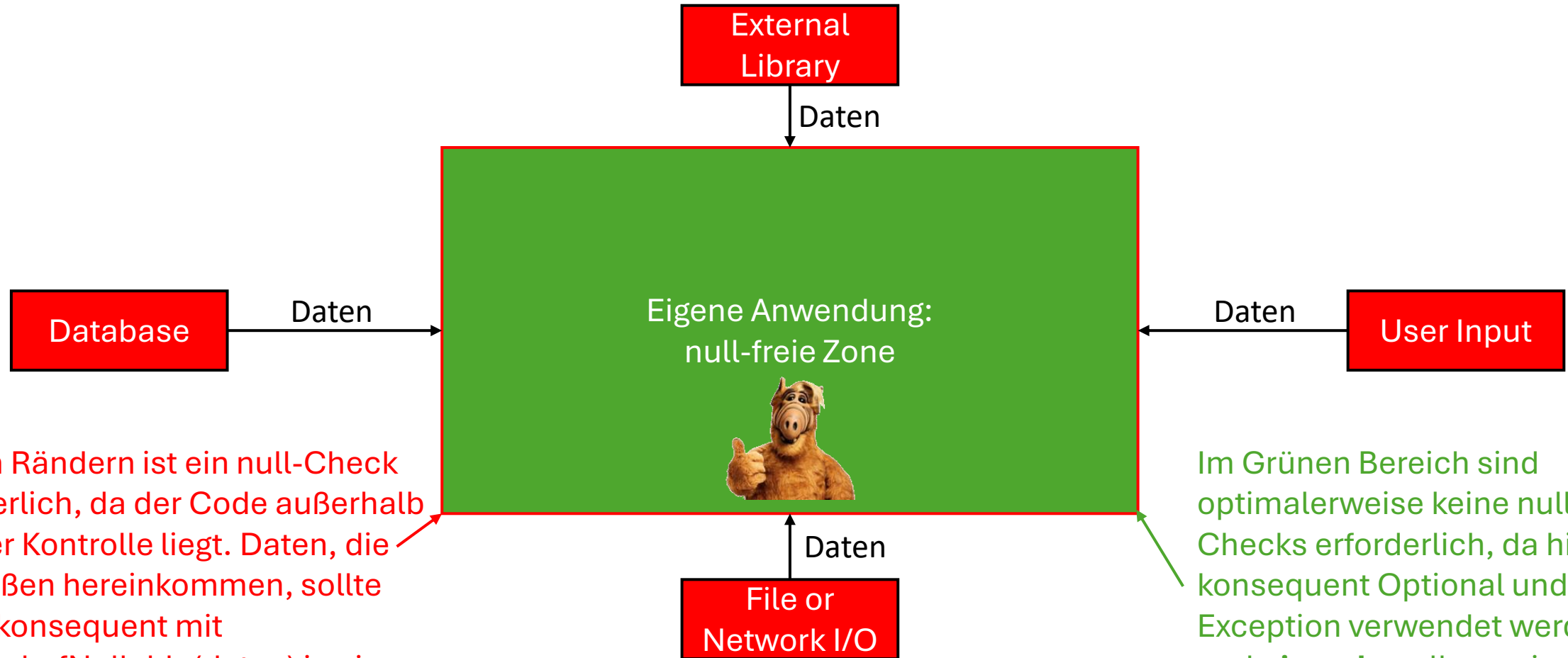
```
public class DatabaseException extends RuntimeException {  
    public DatabaseException(String message) {  
        super(message);  
    }  
}
```



Hier gibt findCustomer ein Optional<Customer> zurück, da eventuell kein Customer mit dem gesuchten Namen gefunden werden kann (was durchaus häufiger vorkommen kann und nichts Ungewöhnliches darstellt).

Außerdem wird eine unchecked DatabaseException geworfen, falls etwas bei der Verbindung zur Datenbank schief geht (was nicht vorkommen sollte, da dies eine weitere Nutzung der Anwendung stark einschränken würde).

To null-check or not to null-check?



An den Rändern ist ein null-Check erforderlich, da der Code außerhalb unserer Kontrolle liegt. Daten, die von Außen hereinkommen, sollte daher konsequent mit `Optional.ofNullable(daten)` in ein `Optional` umgewandelt werden.

Im Grünen Bereich sind optimalerweise keine null-Checks erforderlich, da hier konsequent `Optional` und `Exception` verwendet werden und **niemals** null von einer Methode zurückgegeben wird.